

MODELAGEM E DESENVOLVIMENTO DE BANCO DE DADOS

SEMANA 18

Cláusulas SELECT Avançadas

Funções de janela • CTE • UNION / UNION ALL • CASE

Roteiro completo: aula teórica + atividade prática

Carga horária: 3 aulas de 50 minutos

Ambiente prático: OneCompiler — SQLite

Proposta didática

O material concentra os conceitos essenciais dos quatro conteúdos da semana e adapta a prática para SQLite no OneCompiler. A primeira aula apresenta os conceitos; as duas últimas desenvolvem uma atividade guiada e progressiva, sem exigir MySQL instalado.

PARTE 1 — ROTEIRO DA AULA TEÓRICA

Aula 1 — 50 minutos

1. Objetivos de aprendizagem

- Compreender por que algumas consultas precisam analisar várias linhas sem perder os detalhes individuais.
- Utilizar a ideia de função de janela com ROW_NUMBER(), OVER e PARTITION BY.
- Compreender CTE como uma consulta temporária nomeada criada com WITH.
- Diferenciar UNION de UNION ALL.
- Utilizar CASE para classificar resultados a partir de condições.
- Relacionar os quatro recursos com situações de relatórios e análise de dados.

2. Organização das três aulas

Aula	Tempo	Foco
1	50 min	Teoria: Window Functions, CTE, UNION/UNION ALL e CASE
2	50 min	Prática no OneCompiler: base de dados, ROW_NUMBER(), PARTITION BY e CTE
3	50 min	Prática no OneCompiler: UNION/UNION ALL, CASE, testes e evidências

3. Aula 1 minuto a minuto

- 0–5 min — Situação-problema: como criar relatórios mais inteligentes sem perder os detalhes das linhas?
- 5–17 min — Funções de janela: ROW_NUMBER(), OVER e PARTITION BY.
- 17–29 min — CTE: WITH, legibilidade e ideia de CTE recursiva.
- 29–39 min — UNION e UNION ALL.
- 39–47 min — CASE e classificação de dados.
- 47–50 min — Síntese e preparação para a prática.

4. Funções de janela

Explique que uma função de janela calcula informações considerando outras linhas relacionadas, mas mantém cada registro no resultado. É diferente de um GROUP BY tradicional, que normalmente resume várias linhas em uma só.

Analogia

Imagine uma lista de corredores em uma prova. Cada corredor continua aparecendo individualmente, mas acrescentamos uma coluna mostrando sua posição no ranking. A linha não desaparece; ela ganha uma informação calculada em relação às demais.

```
SELECT
  id_venda,
  data_venda,
  produto,
  vendedor,
```

```
ROW_NUMBER() OVER (ORDER BY data_venda DESC) AS numero_venda
FROM vendas;
```

Explique: ROW_NUMBER() gera números sequenciais; OVER define o conjunto e a ordem usados pelo cálculo; ORDER BY dentro de OVER controla a numeração.

5. PARTITION BY

PARTITION BY divide logicamente os resultados em grupos para a função de janela. A numeração pode, por exemplo, recomeçar para cada vendedor.

```
SELECT
    id_venda,
    vendedor,
    valor_total,
    ROW_NUMBER() OVER (
        PARTITION BY vendedor
        ORDER BY valor_total DESC
    ) AS posicao
FROM vendas;
```

Ponto importante

PARTITION BY não cria novas tabelas e não divide fisicamente o banco. Ele apenas define grupos lógicos para a função de janela.

6. CTE — Common Table Expression

Uma CTE é uma consulta temporária nomeada, criada com WITH, que existe apenas durante a execução da instrução. Ela ajuda a dividir uma consulta maior em etapas mais fáceis de ler.

Analogia

Uma CTE pode ser comparada a um rascunho com nome. Primeiro você calcula uma parte do problema, dá um nome a esse resultado e depois utiliza esse nome na consulta principal.

```
WITH total_vendas AS (
    SELECT
        vendedor,
        SUM(valor_total) AS total
    FROM vendas
    GROUP BY vendedor
)
SELECT vendedor, total
FROM total_vendas
ORDER BY total DESC;
```

7. CTE recursiva — conceito

Apresente apenas a ideia: uma CTE recursiva pode percorrer estruturas hierárquicas, como supervisor → funcionário → subordinado. Como o tema é mais avançado, a prática principal trabalhará primeiro com CTE simples.

```
WITH RECURSIVE hierarquia AS (  
    -- ponto inicial  
    SELECT id_funcionario, nome, id_supervisor, 1 AS nivel  
    FROM funcionarios  
    WHERE id_supervisor IS NULL  
  
    UNION ALL  
  
    -- próximos níveis  
    SELECT f.id_funcionario, f.nome, f.id_supervisor, h.nivel + 1  
    FROM funcionarios f  
    INNER JOIN hierarquia h  
        ON f.id_supervisor = h.id_funcionario  
)  
SELECT * FROM hierarquia;
```

8. UNION × UNION ALL

Os dois operadores combinam os resultados de consultas compatíveis.

```
SELECT nome FROM clientes  
UNION  
SELECT nome FROM fornecedores;
```

UNION remove linhas duplicadas.

```
SELECT nome FROM clientes  
UNION ALL  
SELECT nome FROM fornecedores;
```

UNION ALL mantém todas as ocorrências, inclusive duplicadas.

Regra prática

Use UNION quando o resultado precisa ser único. Use UNION ALL quando as repetições são importantes ou quando você sabe que não haverá duplicações.

9. CASE

CASE cria resultados condicionais dentro da consulta. Ele funciona de forma parecida com uma estrutura if/elif/else.

```
SELECT  
    cliente,  
    valor_total,  
    CASE  
        WHEN valor_total > 1000 THEN 'Alta'
```

```
        WHEN valor_total BETWEEN 500 AND 1000 THEN 'Media'
        ELSE 'Baixa'
    END AS categoria_venda
FROM vendas;
```

Explique WHEN = condição; THEN = resultado quando verdadeira; ELSE = valor padrão; END = final da expressão.

10. Conteúdo para a lousa

WINDOW FUNCTION

Mantém as linhas e acrescenta um cálculo.

ROW_NUMBER()

Numera linhas.

OVER(...)

Define como a função de janela será aplicada.

PARTITION BY

Cria grupos lógicos para a função de janela.

CTE = WITH

Consulta temporária nomeada.

UNION

Une resultados e remove duplicatas.

UNION ALL

Une resultados e mantém duplicatas.

CASE

Cria classificações condicionais.

11. Perguntas rápidas para verificar a compreensão

1. ROW_NUMBER() elimina linhas da consulta?
2. Para que serve PARTITION BY?
3. Uma CTE fica salva permanentemente no banco?
4. Qual é a diferença principal entre UNION e UNION ALL?
5. Em um CASE, para que serve WHEN?
6. Por que funções de janela são úteis em rankings?

PARTE 2 — ROTEIRO DA ATIVIDADE PRÁTICA

Aulas 2 e 3 — 100 minutos

1. Identificação

Nome	Turma	Data
Arthur Saraiva de França	3b	26/08/2026

2. Contexto

Você faz parte da equipe de dados de uma empresa que precisa produzir relatórios mais claros sobre vendas, vendedores, clientes e produtos. O objetivo é utilizar recursos avançados de SELECT para numerar resultados, organizar cálculos, combinar conjuntos de dados e classificar informações.

3. Ambiente — OneCompiler com SQLite

- Acesse o OneCompiler.
- Escolha a opção SQLite.
- Apague o código de exemplo existente.
- Cole primeiro o script de criação das tabelas e execute.
- Depois execute cada consulta separadamente para observar o resultado.

Importante

Os exemplos abaixo foram adaptados para SQLite. Não use CREATE DATABASE, USE ou recursos exclusivos do MySQL. ROW_NUMBER(), WITH/CTE, WITH RECURSIVE, UNION, UNION ALL e CASE são compatíveis com a versão atual do SQLite usada para esta adaptação.

AULA 2 — PARTE 1 — 50 MINUTOS

Objetivo: montar a base e praticar funções de janela e CTE.

Etapa 1 — Criar a tabela de vendas

```
CREATE TABLE vendas (  
  id_venda INTEGER PRIMARY KEY,  
  data_venda TEXT NOT NULL,  
  produto TEXT NOT NULL,  
  vendedor TEXT NOT NULL,  
  cliente TEXT NOT NULL,  
  regioao TEXT NOT NULL,  
  quantidade INTEGER NOT NULL,  
  valor_total REAL NOT NULL  
);
```

Etapa 2 — Inserir os dados

```
INSERT INTO vendas VALUES  
(1, '2024-10-01', 'Laptop', 'Carlos', 'Ana Souza', 'Sul', 2, 3000.00),  
(2, '2024-10-05', 'Smartphone', 'Ana', 'Bruno Lima', 'Sul', 1, 1500.00),  
(3, '2024-10-06', 'Tablet', 'Carlos', 'Carla Dias', 'Norte', 3, 1800.00),  
(4, '2024-10-07', 'Monitor', 'Bruno', 'Diego Alves', 'Norte', 4, 1200.00),  
(5, '2024-10-08', 'Laptop', 'Ana', 'Eva Martins', 'Sul', 1, 1500.00),
```

```
(6, '2024-10-09', 'Smartphone', 'Bruno', 'Felipe Rocha', 'Norte', 2,3000.00),
(7, '2024-10-10', 'Monitor', 'Carlos', 'Gabriela Nunes', 'Sul', 1,600.00),
(8, '2024-10-12', 'Tablet', 'Ana', 'Helena Costa', 'Norte', 5,3000.00),
(9, '2024-10-13', 'Smartphone', 'Carlos', 'Igor Melo', 'Sul', 2,3000.00),
(10, '2024-10-15', 'Laptop', 'Bruno', 'Joana Reis', 'Norte', 2,4000.00);
```

Execute para conferir:

```
SELECT * FROM vendas;
```

Etapa 3 — ROW_NUMBER()

Gere uma numeração para as vendas, começando pela mais recente.

```
SELECT
    id_venda,
    data_venda,
    produto,
    vendedor,
    ROW_NUMBER() OVER (ORDER BY data_venda DESC) AS numero_venda
FROM vendas;
```

Observe: qual venda recebeu o número 1?

Venda **10**, do dia **2024-10-15**.

Etapa 4 — PARTITION BY

Agora a numeração deve reiniciar para cada vendedor.

```
SELECT
    id_venda,
    vendedor,
    valor_total,
    ROW_NUMBER() OVER (
        PARTITION BY vendedor
        ORDER BY valor_total DESC
    ) AS posicao
FROM vendas
ORDER BY vendedor, posicao;
```

O que mudou em relação à consulta anterior?

A numeração passou a reiniciar para cada vendedor.

Etapa 5 — CTE simples

Use uma CTE para calcular o total vendido por cada vendedor.

```
WITH total_vendas AS (
    SELECT
        vendedor,
        SUM(valor_total) AS total
    FROM vendas
    GROUP BY vendedor
)
SELECT vendedor, total
FROM total_vendas
```

```
ORDER BY total DESC;
```

Explique com suas palavras o que a CTE `total\_vendas` representa:

Representa o total de vendas realizadas por cada vendedor.

Evidência 1 — Aula 2

Insira um print mostrando uma consulta com ROW_NUMBER() ou PARTITION BY funcionando.

id_venda	data_venda	produto	vendedor	cliente	regiao	quantidade	valor_total
1	2024-10-01	Laptop	Carlos	Ana Souza	Sul	2	3000.0
2	2024-10-05	Smartphone	Ana	Bruno Lima	Sul	1	1500.0
3	2024-10-06	Tablet	Carlos	Carla Dias	Norte	3	1800.0
4	2024-10-07	Monitor	Bruno	Diego Alves	Norte	4	1200.0
5	2024-10-08	Laptop	Ana	Eva Martins	Sul	1	1500.0
6	2024-10-09	Smartphone	Bruno	Felipe Rocha	Norte	2	3000.0
7	2024-10-10	Monitor	Carlos	Gabriela Nunes	Sul	1	600.0
8	2024-10-12	Tablet	Ana	Helena Costa	Norte	5	3000.0
9	2024-10-13	Smartphone	Carlos	Igor Melo	Sul	2	3000.0
10	2024-10-15	Laptop	Bruno	Joana Reis	Norte	2	4000.0

AULA 3 — PARTE 2 — 50 MINUTOS

Objetivo: comparar UNION e UNION ALL e aplicar CASE na análise de vendas.

Etapa 6 — Criar tabelas para UNION

```
CREATE TABLE clientes (  
    nome TEXT,  
    email TEXT  
);  
  
INSERT INTO clientes VALUES  
( 'Ana Souza', 'ana@email.com'),  
( 'Bruno Lima', 'bruno@email.com'),  
( 'Carla Dias', 'carla@email.com');  
  
CREATE TABLE fornecedores (  
    nome TEXT,  
    email TEXT  
);  
  
INSERT INTO fornecedores VALUES  
( 'Carla Dias', 'carla@email.com'),  
( 'Delta Tech', 'contato@delta.com'),  
( 'Ana Souza', 'ana@email.com');
```

Etapa 7 — UNION

Execute a consulta:

```
SELECT nome FROM clientes  
UNION  
SELECT nome FROM fornecedores  
ORDER BY nome;
```

Quantos nomes foram exibidos? 4 nomes.

Etapa 8 — UNION ALL

```
SELECT nome FROM clientes  
UNION ALL  
SELECT nome FROM fornecedores  
ORDER BY nome;
```

O que aconteceu com Ana Souza e Carla Dias?

Elas apareceram **duas vezes**, pois o UNION ALL mantém as duplicatas.

Etapa 9 — CASE: classificação das vendas

Adapte a proposta da atividade MarketData Solutions e classifique as vendas.

```
SELECT  
    cliente,  
    valor_total,  
    CASE  
        WHEN valor_total > 1000 THEN 'Alta'  
        WHEN valor_total BETWEEN 500 AND 1000 THEN 'Media'
```

```

        ELSE 'Baixa'
    END AS categoria_venda
FROM vendas
ORDER BY id_venda;

```

Escolha duas linhas do resultado e registre a categoria obtida:

Ana Souza — Alta

Bruno Lima — Alta

Etapa 10 — CASE: desempenho dos vendedores

Agora combine CTE + CASE. Primeiro, a CTE soma as vendas; depois, CASE classifica o desempenho.

```

WITH total_vendedores AS (
    SELECT
        vendedor,
        SUM(valor_total) AS total_vendas
    FROM vendas
    GROUP BY vendedor
)
SELECT
    vendedor,
    total_vendas,
    CASE
        WHEN total_vendas > 10000 THEN 'Excelente'
        WHEN total_vendas BETWEEN 5000 AND 10000 THEN 'Bom'
        ELSE 'Regular'
    END AS desempenho
FROM total_vendedores
ORDER BY total_vendas DESC;

```

Qual é a vantagem de usar a CTE antes do CASE?

A CTE organiza o cálculo do total antes de fazer a classificação.

Etapa 11 — CASE: classificação de produtos

Crie a tabela:

```

CREATE TABLE produtos (
    id_produto INTEGER PRIMARY KEY,
    nome_produto TEXT NOT NULL,
    preco REAL NOT NULL
);

INSERT INTO produtos VALUES
(1, 'Mouse', 45.00),
(2, 'Teclado', 80.00),
(3, 'Headset', 120.00),
(4, 'Monitor', 900.00);

```

Agora classifique os preços usando CASE:

```

SELECT
    nome_produto,
    preco,
    CASE

```

```

        WHEN preco > 100 THEN 'Caro'
        WHEN preco BETWEEN 50 AND 100 THEN 'Moderado'
        ELSE 'Barato'
    END AS categoria_preco
FROM produtos
ORDER BY preco;

```

Etapa 12 — Desafio opcional: CTE recursiva

Faça apenas se a turma estiver acompanhando bem. O objetivo é observar uma hierarquia.

```

CREATE TABLE funcionarios (
    id_funcionario INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    id_supervisor INTEGER
);

```

```

INSERT INTO funcionarios VALUES
(1, 'Carlos', NULL),
(2, 'Maria', 1),
(3, 'Joao', 1),
(4, 'Ana', 2),
(5, 'Pedro', 3);

```

```

WITH RECURSIVE hierarquia AS (
    SELECT
        id_funcionario,
        nome,
        id_supervisor,
        1 AS nivel
    FROM funcionarios
    WHERE id_supervisor IS NULL

    UNION ALL

    SELECT
        f.id_funcionario,
        f.nome,
        f.id_supervisor,
        h.nivel + 1
    FROM funcionarios f
    INNER JOIN hierarquia h
        ON f.id_supervisor = h.id_funcionario
)
SELECT *
FROM hierarquia
ORDER BY nivel, id_funcionario;

```

Não é obrigatório

Se a turma apresentar dificuldade com CTE simples, não avance para a recursiva. O objetivo principal da semana pode ser alcançado com ROW_NUMBER(), CTE simples, UNION/UNION ALL e CASE.

Evidência 2 — UNION / UNION ALL

Insira um print comparando os dois resultados.

nome
Ana Souza
Ana Souza
Bruno Lima
Carla Dias
Carla Dias
Delta Tech

Evidência 3 — CASE

Insira um print mostrando uma classificação com CASE.

nome
Ana Souza
Bruno Lima
Carla Dias
Delta Tech

nome
Ana Souza
Ana Souza
Bruno Lima
Carla Dias

13. Questões finais

1. Qual é a função de ROW_NUMBER()?

Numerar as linhas do resultado.

2. Para que serve PARTITION BY?

Para dividir os dados em grupos e reiniciar o cálculo em cada grupo.

3. O que uma CTE ajuda a organizar?

Ajuda a organizar consultas complexas em etapas.

4. Qual é a diferença entre UNION e UNION ALL?

UNION remove duplicatas; UNION ALL mantém as duplicatas.

5. Para que serve CASE em uma consulta SQL?

Serve para criar classificações e condições nos resultados.

14. Checklist

- ☐ Abri o OneCompiler com SQLite.
- ☐ Criei e consultei a tabela vendas.
- ☐ Executei ROW_NUMBER().
- ☐ Utilizei PARTITION BY.
- ☐ Executei uma CTE simples.
- ☐ Comparei UNION e UNION ALL.
- ☐ Utilizei CASE para classificar vendas.
- ☐ Utilizei CASE para classificar desempenho.

- ☐ Utilizei CASE para classificar produtos.
- ☐ Registre os prints solicitados.
- ☐ Respondi às questões finais.

15. Critérios de avaliação — 10 pontos

Critério	Valor
Criação e uso da base de dados	1,0
ROW_NUMBER() e OVER	1,5
PARTITION BY	1,5
CTE simples	1,5
UNION e UNION ALL	1,5
CASE e classificações	2,0
Evidências, questões e organização	1,0

PARTE 3 — ORIENTAÇÕES AO PROFESSOR

1. Condução da Aula 2

- Faça os alunos executarem primeiro `SELECT * FROM vendas` para confirmar que a base foi criada corretamente.
- No `ROW_NUMBER()`, peça que identifiquem visualmente a venda mais recente antes de executar.
- No `PARTITION BY`, destaque que a numeração reinicia em cada vendedor.
- Na CTE, apresente o conteúdo entre parênteses como uma etapa intermediária que recebe um nome.
- Evite avançar para CTE recursiva se a maioria ainda não compreendeu `WITH` simples.

2. Condução da Aula 3

- Antes de `UNION`, peça que os estudantes identifiquem nomes repetidos entre clientes e fornecedores.
- Execute `UNION` e `UNION ALL` em sequência e peça comparação visual.
- No `CASE`, peça que os estudantes leiam as condições de cima para baixo.
- Destaque que a ordem dos `WHEN` pode alterar a classificação.
- Use a atividade MarketData Solutions como aplicação final do `CASE` em relatórios.

3. Erros comuns

Erro: no such table

A tabela ainda não foi criada na execução atual. Execute primeiro o script de criação e inserção.

Erro de sintaxe perto de OVER

Confira parênteses, vírgulas e a estrutura `ROW_NUMBER() OVER (...)`.

PARTITION BY não reinicia como esperado

Confira se a coluna usada em `PARTITION BY` é realmente o agrupamento desejado.

UNION dá erro de quantidade de colunas

As consultas combinadas precisam retornar a mesma quantidade de colunas compatíveis.

CASE classifica errado

Leia os `WHEN` na ordem. A primeira condição verdadeira é utilizada.

CTE não é encontrada fora da consulta

Explique que ela existe somente durante a instrução iniciada por `WITH`.

4. Validação técnica realizada

Os scripts utilizados neste roteiro foram verificados em SQLite: criação das tabelas, `INSERT`, `ROW_NUMBER()`, `OVER`, `PARTITION BY`, CTE simples, `WITH RECURSIVE`, `UNION`, `UNION ALL` e `CASE`. A sintaxe foi mantida compatível com a proposta de execução no OneCompiler usando SQLite.

5. Síntese da Semana 18

Síntese

Funções de janela acrescentam análises sem eliminar as linhas individuais; CTEs organizam consultas em etapas nomeadas; `UNION` e `UNION ALL` combinam conjuntos de resultados com comportamentos diferentes para duplicatas; `CASE` transforma regras de negócio em classificações dentro do `SELECT`.